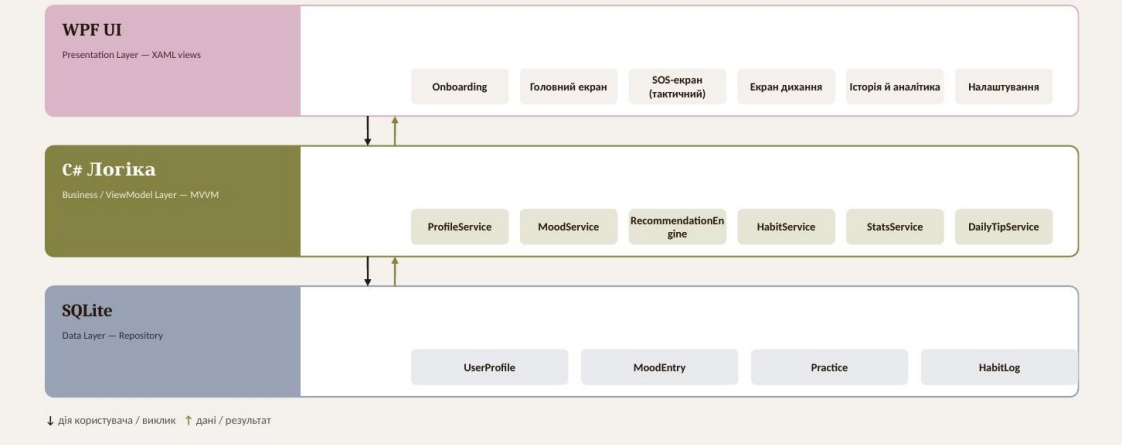
1. Схема основних потоків взаємодії

Схема нижче показує, які компоненти належать кожному шару та в якому напрямку рухаються дії користувача і дані. Стрілка вниз — дія користувача або виклик команди; стрілка вгору — дані або результат, що повертаються в інтерфейс.

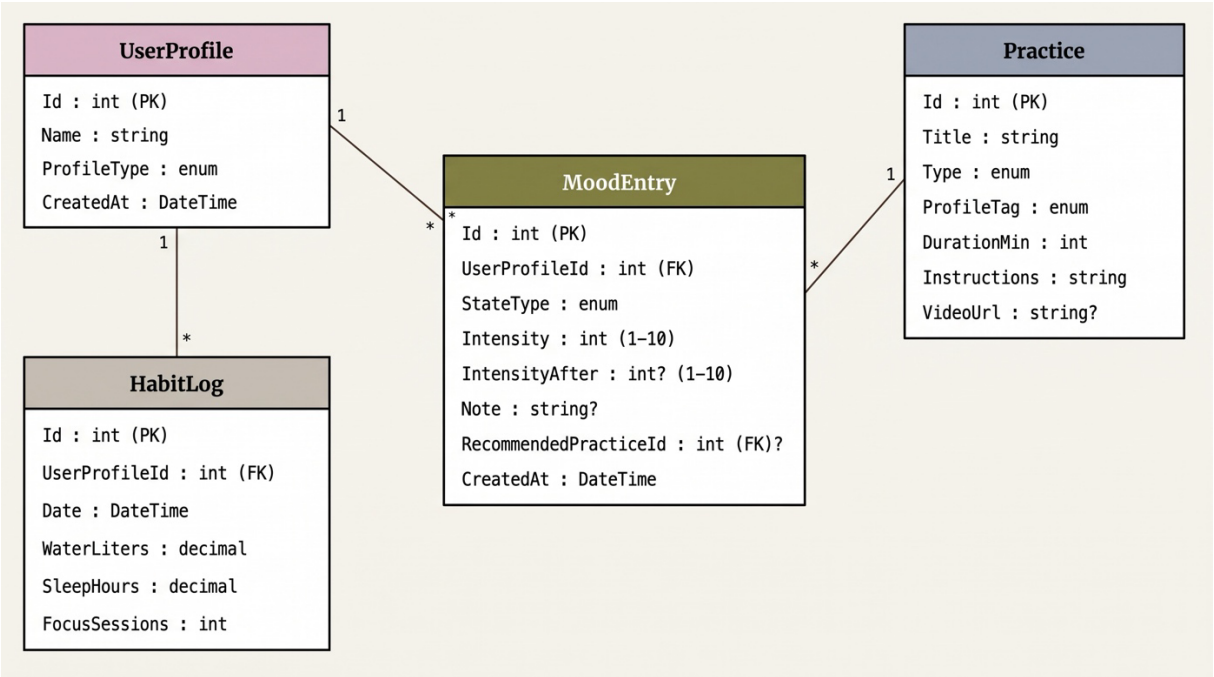


Мал. 1 — Схема основних потоків взаємодії WPF ↔ C# ↔ SQLite

Presentation Layer об'єднує всі екрани застосунку: онбординг, головний екран (для кожного профілю — свій набір блоків), SOS-екран тактичного режиму, екран дихальної практики, історію й аналітику та налаштування. Business Layer реалізований через шість сервісів: ProfileService, MoodService, RecommendationEngine, HabitService, StatsService та DailyTipService. Data Layer зберігає чотири основні сутності: UserProfile, MoodEntry, Practice та HabitLog.

2. UML Class Diagram — модель даних

Use Case і Sequence діаграми показують поведінку системи, але не структуру даних. Діаграма класів нижче деталізує чотири сутності SQLite, їхні поля та зв'язки між ними.

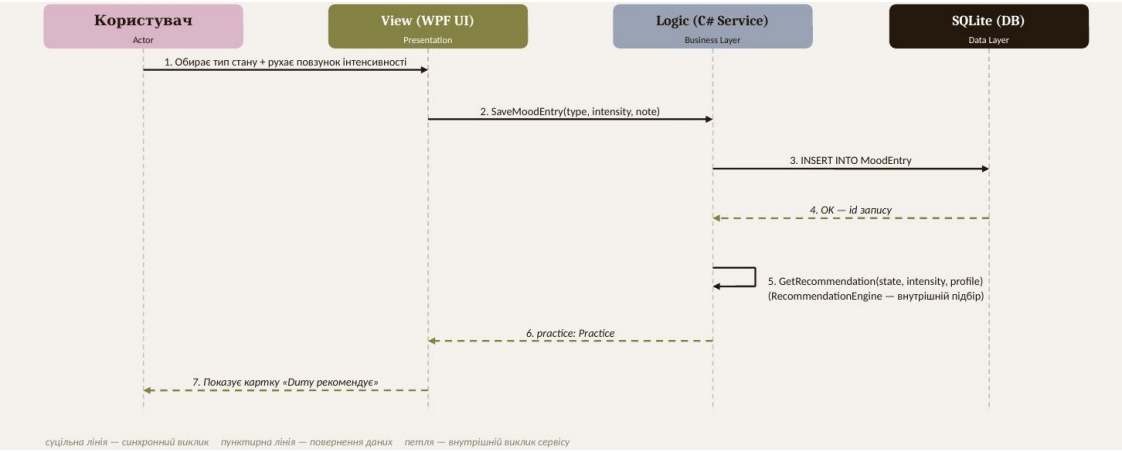


Мал. 2 — UML Class Diagram моделі даних Dumy

UserProfile зберігає обраний профіль користувача і пов'язаний зв'язком «один до багатьох» із MoodEntry (записи стану) та HabitLog (щоденні звички). Practice пов'язана з MoodEntry необов'язковим зв'язком «один до багатьох»: кожен запис стану може містити посилання на рекомендовану практику (RecommendedPracticeId), що дозволяє показувати історію виконаних практик на екрані аналітики. **В оновленій моделі до MoodEntry додано поле IntensityAfter для оцінки стану після виконання вправи, а до Practice — поле VideoUrl для посилань на відеоматеріали (наприклад, з йоги).**

3. UML Sequence Diagram — основний сценарій

Діаграма послідовності деталізує сценарій US-01 «Зафіксувати стан та отримати рекомендацію» — шлях, який користувач проходить найчастіше. У діаграмі задіяно чотири учасники: Користувач (актор), View (WPF UI), Logic (C# Service) та SQLite (DB).



Мал. 3 — UML Sequence Diagram сценарію фіксації стану та отримання рекомендації

Користувач обирає тип стану і рухає повзунок інтенсивності (1) — це передається у View, який викликає команду SaveMoodEntry у шарі Logic (2). Logic виконує запис у базу через SQL-команду INSERT INTO MoodEntry (3) і отримує від бази підтвердження та ідентифікатор запису (4). Після успішного збереження Logic виконує внутрішній виклик GetRecommendation (5) — самопетля всередині RecommendationEngine, яка підбирає практику за станом, інтенсивністю й профілем. Результат — об'єкт Practice — повертається у View (6), і View одразу показує користувачу картку «Dumy рекомендує» (7).

Висновок

Обраний триярусний підхід з MVVM відповідає ключовій вимозі проєкту — швидкій адаптації інтерфейсу під три профілі без дублювання коду — і водночас лишає бізнес-логіку тестованою окремо від UI. Схема потоків взаємодії, діаграма класів і діаграма послідовності разом дають повний комплект моделей, необхідний для переходу до реалізації MVP.